

Modelos de Aprendizaje Automático Clásico

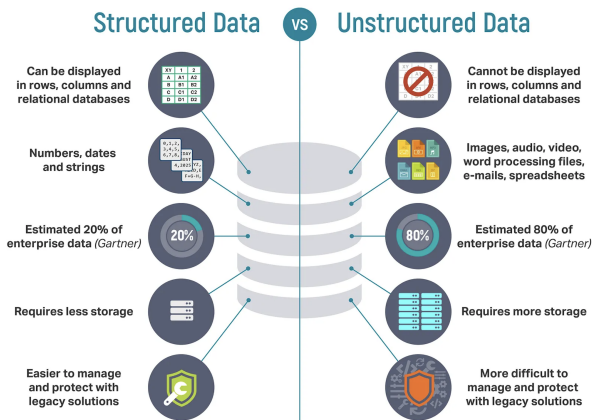
Julián Gil González, Ph.D

Modelos de Predicción y Aprendizaje Automático Informado por la Física Aplicado a
Sistemas Energéticos.

2026



Introducción



Tomado de: <https://lawtomated.com/structured-data-vs-unstructured-data-what-are-they-and-why-care/>

Agenda

1. Modelos Lineales

2. Máquinas de Vectores de Soporte

3. Modelos Basado en Árboles

4. Redes Neuronales Profundas

5. Revisitando la Regresión Lineal

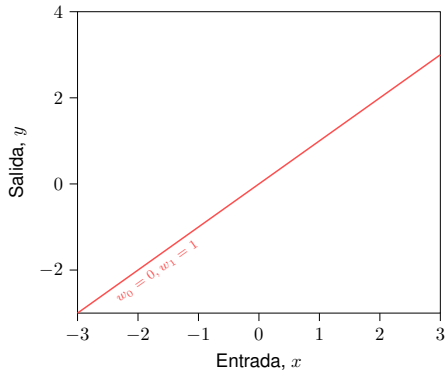
Modelo Regresión lineal

□ Modelo:

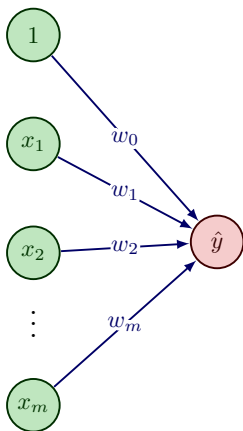
$$\begin{aligned}y &= f(x, \mathbf{w}) \\ &= w_0 + w_1 x\end{aligned}$$

□ Parámetros:

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$



Regresión Lineal

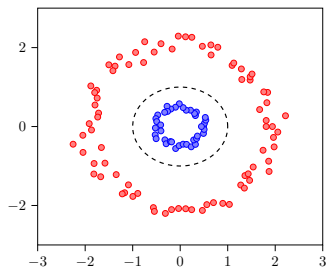
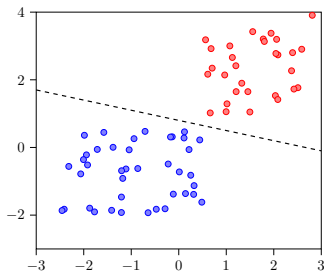
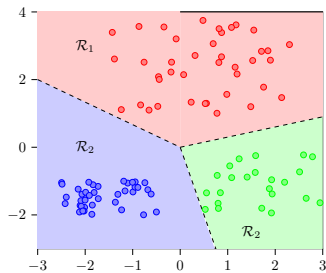
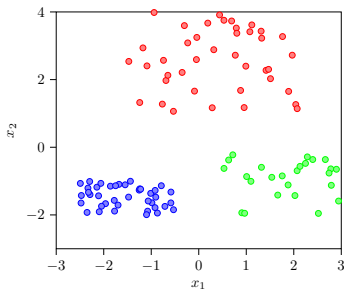


- La regresión lineal puede ser aplicada a datos con múltiples atributos.
- En este caso, la predicción se calcula como **combinación lineal** de todos los atributos de entrada.

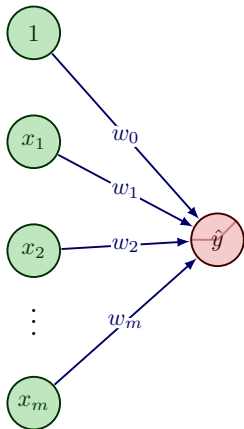
$$\hat{y} = w_0 + w_1x_1 + \cdots + w_mx_m$$

- Los parámetros del modelo se estiman a partir de los conceptos de **Función de Pérdida** y **gradiente descendente**.

Clasificación



Regresión Logística (Clasificación binaria)

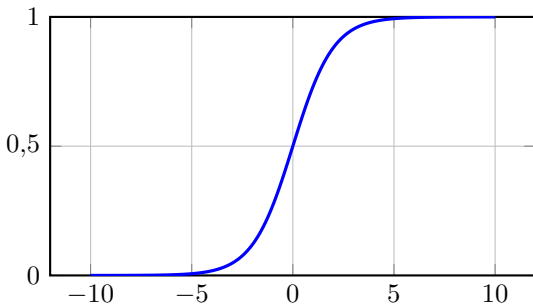


- La regresión logística realiza predicciones a partir de una **combinación lineal** de los atributos.
- En la regresión logística se mide la probabilidad de que una instancia pertenezca a una de las dos clases (clasificación binaria).
- La **combinación lineal** toma cualquier valor real. Así, para darle sentido de probabilidad a la predicción, se usa la función **Sigmoidal**, $\sigma(\cdot)$.

$$\hat{y} = p(y = 1|x) = \sigma \left(w_0 + \sum_{j=1}^m w_j * x_j \right)$$

Regresión Logística (II)

$$\sigma(a) = \frac{1}{1+\exp(-a)}$$



- La función de verosimilitud se define como (Distribución de Bernoulli)

$$p(\mathbf{y}|\mathbf{w}) = \prod_{n=1}^N \hat{y}_n^{y_n} (1 - \hat{y}_n)^{1-y_n}, \quad \hat{y}_n = p(\mathcal{C}_1|\mathbf{x}_n) = \sigma(\mathbf{w}^\top \mathbf{x}).$$

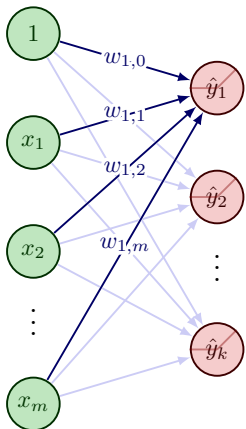
Regresión Logística (III)

- Los parámetros del modelo $\mathbf{w}$ se estiman a partir de la minimización del logaritmo negativo de la verosimilitud.

$$\mathcal{L}(\mathbf{w}) = -\log p(\mathbf{y}|\mathbf{w}) = -\sum_{n=1}^N y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n).$$

- A esta función se la conoce como la Entropía Cruzada.

Regresión Logística (Clasificación multiclase)



- La **regresión logística** pueden extenderse para problemas de clasificación de múltiples clases.
- Se deben definir K salidas, cada salida está asociada con una de las clases.
- Con el fin de darle sentido de probabilidad a las predicciones, se usa la función **Softmax**, la cual transforma la salida de la combinación lineal a valores entre 0 y 1.

$$\hat{y}_k = p(y = k|x), \quad \sum_{k=1}^K \hat{y}_k = 1.$$

Agenda

1. Modelos Lineales

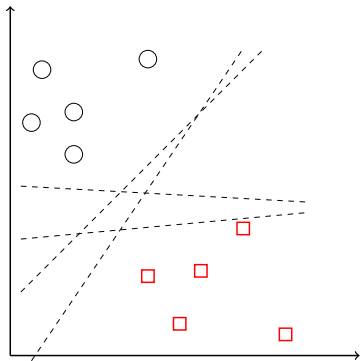
2. Máquinas de Vectores de Soporte

3. Modelos Basado en Árboles

4. Redes Neuronales Profundas

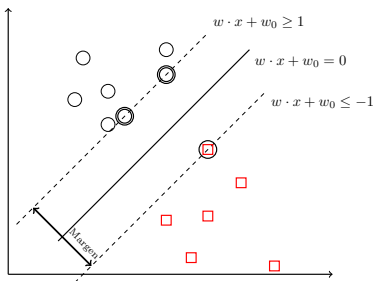
5. Revisitando la Regresión Lineal

Fundamentos de las SVMs: Márgenes duros



- En clasificación binaria la idea es dividir el espacio de entrada en dos regiones.
- En clasificación por SVM se considera que $y_n \in \{-1, 1\}$.
- Suponer que los datos son linealmente separables.
- Existen infinitas rectas (o hiperplanos) que dividen el conjunto de datos en dos.

Fundamentos de las SVMs: Márgenes duros



- Se puede emplear el modelo lineal $\mathbf{x}_n^\top \mathbf{w} + w_0$. Así, la predicción es $\hat{y}_n = \text{sgn}(\mathbf{x}_n^\top \mathbf{w} + w_0)$.
- Las SVMs seleccionan la recta que maximiza el margen de separación entre las clases.
- El margen se define como la distancia del hiperplano al punto más cercano en cada clase.
- Los puntos más cercanos a la superficie de separación se conocen como los vectores de soporte.

Problema de optimización

- Maximizar el margen equivale a minimizar $\|\mathbf{w}\|^2$:

$$\arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

- El problema de optimización está sujeto a las siguientes restricciones:

$$y_n (\mathbf{x}_n^\top \mathbf{w} + w_0) \geq 1, \quad n = 1, \dots, N$$

- Los puntos que cumplen la igualdad son restricciones activas, los demás son inactivas. Siempre hay al menos un punto activo (el más cercano al hiperplano).

Problema Dual

- a partir de las condiciones previas se establece el siguiente problema dual.

$$\max_{\mathbf{a}} \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m y_n y_m \mathbf{x}_n^{\top} \mathbf{x}_m,$$

sujeto a

$$a_n \geq 0, \quad \sum_{n=1}^N y_n a_n = 0,$$

donde $\mathbf{a} = [a_1, \dots, a_N]$ indica los vectores de soporte.

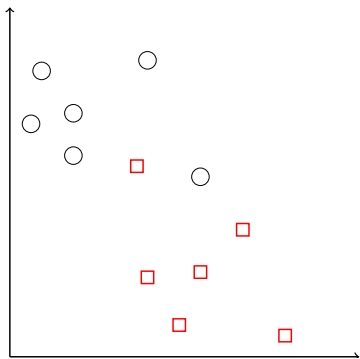
Predicciones

- Una vez se determinan los valores del vector $\mathbf{a}$ y dada una muestra del conjunto de prueba $\mathbf{x}_*$, se puede predecir su etiqueta como:

$$y(\mathbf{x}_*) = \sum_{n=1}^N a_n y_n \mathbf{x}_n^\top \mathbf{x}_* + w_0.$$

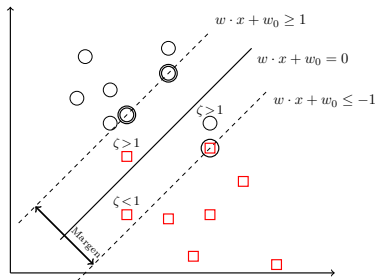
- Notemos que cuando el valor $a_n = 0$, la muestra $\mathbf{x}_n$ no participa en el cálculo de la predicción.
- Por el contrario, cuando $a_n > 0$, la muestra $\mathbf{x}_n$ se considera como un *vector de soporte* y contribuye al cálculo de las predicciones.

SVMs con márgenes suaves: clases traslapadas



- La optimización de las SVMs con márgenes duros no tiene solución si los datos no son linealmente separables.
- En general, es difícil encontrar datos que sean linealmente separables.
- Es más común encontrar datos, como los de la figura, donde la estructura es principalmente lineal, pero existen algunos datos que se traslapan.

Variables de Holgura



- Una solución para esta condición, es la inclusión de un conjunto de **variables de holgura** $\zeta_n \geq 0$.
- Las muestras con $\zeta_n = 0$ están correctamente clasificadas y se encuentran fuera del margen.
- Para $0 < \zeta_n \leq 1$, la muestra es clasificada correctamente, pero se encuentra dentro de la margen.
- Finalmente, un $\zeta_n > 1$ indica que la muestra está en el lado incorrecto del hiperplano y por lo tanto se clasificará de forma errónea.

Problema de optimización

- Al incluir las variables de holgura, el problema de optimización se puede reescribir como:

$$\min_{\mathbf{w}, w_0, \zeta} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \zeta_n$$

sujeto a

$$\zeta_n \geq 0$$

$$y_n(\mathbf{w}^\top \mathbf{x}_n + w_0) \geq 1 - \zeta_n, \quad n = 1, \dots, N,$$

donde C es un hiperparámetro que controla cuántas muestras pueden violar el margen; cuando $C \rightarrow \infty$, volvemos a la formulación original de las SVMs. Por otro lado, $\zeta = [\zeta_1, \dots, \zeta_N]$.

Problema Dual

- Al igual que con el planteamiento original de las SVMs, se usan los multiplicadores de Lagrange para formular el siguiente “problema dual”

$$\max_{\mathbf{a}} \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m y_n y_m \mathbf{x}_n^\top \mathbf{x}_m,$$

sujeto a

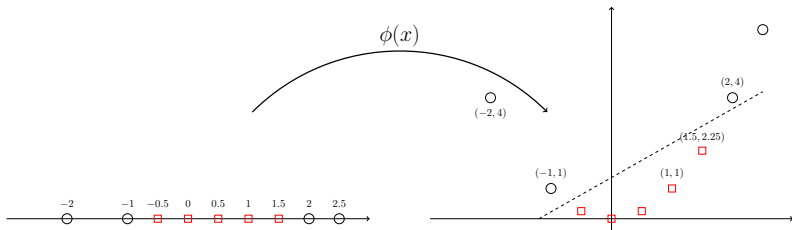
$$0 \leq a_n \leq C, \quad \sum_{n=1}^N y_n a_n = 0$$

donde $\mathbf{a} = [a_1, \dots, a_N]$

SVMs para datos que no son linealmente separables

- Las formulaciones vistas hasta el momento se aplican para datos con estructuras lineales. Sin embargo, es común encontrar datos con estructuras no lineales.
- Por ejemplo, en la figura se muestra un conjunto de datos de una dimensión que no pueden separarse a partir de rectas o hiperplanos.
- Una alternativa es realizar una transformación no lineal w (funciones base) de los datos a un espacio de mayor dimensión con el fin de mejorar su representación.

Funciones Base



- Para el ejemplo, vamos a realizar la siguiente transformación no lineal para obtener un espacio de dos dimensiones $w(x) = [x, x^2]$.
- De esta forma, como se muestra en la figura, la transformación produce datos que son linealmente separables.

Funciones Base

- El teorema de Cover establece que si a un conjunto de datos se le aplica una transformación no-lineal a un espacio lo suficientemente grande, los datos transformados tienen alta probabilidad de ser linealmente separable.
- En este sentido, los datos transformados pueden ser clasificados a partir de rectas.
- Por ejemplo, el problema dual para las SVMs con margen suave se escribe como

$$\max_a \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m y_n y_m w(\mathbf{x}_n)^\top w(\mathbf{x}_m).$$

El truco del kernel

- Una función kernel es una representación del producto punto de los datos transformados. De esta forma,

$$\varphi(\mathbf{x}_n)^\top \varphi(\mathbf{x}_m) = \kappa(\mathbf{x}_n, \mathbf{x}_m).$$

- Así, el problema dual para las SVMs con margen suave y con función kernel se escribe como:

$$\max_{\mathbf{a}} \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m y_n y_m \kappa(\mathbf{x}_n, \mathbf{x}_m).$$

- De igual forma, para la ecuación predictiva tenemos:

$$y(\mathbf{x}_*) = \sum_{n=1}^N a_n y_n \kappa(\mathbf{x}_n, \mathbf{x}_*) + w_0.$$

Funciones Kernel

Existen diversas funciones kernel, algunas de ellas son:

Función de Kernel	Descripción
Kernel Polinomial	$\kappa(x_i, x_j) = (x_i^T x_j + 1)^d$
Kernel Gaussiano	$\kappa(x_i, x_j) = \exp(-\ x_i - x_j\ ^2 / 2\sigma^2)$
Kernel Sigmoidal	$\kappa(x_i, x_j) = \tanh(B_0 x_i^T x_j + B_1)$

Los valores d , σ^2 , B_0 y B_1 se denominan hiperparámetros y deben ser elegidos por el usuario (Búsqueda de Hiperparámetros).

Agenda

1. Modelos Lineales

2. Máquinas de Vectores de Soporte

3. Modelos Basado en Árboles

4. Redes Neuronales Profundas

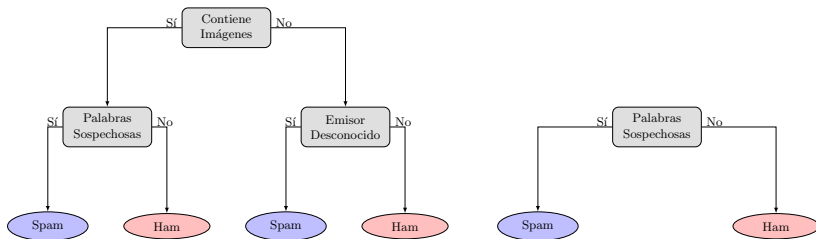
5. Revisitando la Regresión Lineal

Ejemplo de detección de Spam

Un conjunto de datos para la predicción de spam en correos electrónicos.

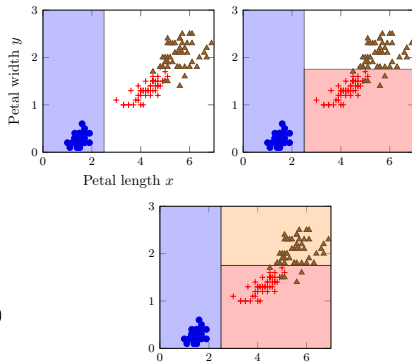
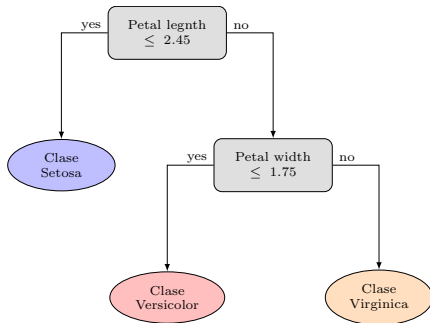
ID	Palabras sospechosas	Remitente desconocido	Contiene imágenes	Clase
376	Sí	No	Sí	spam
489	Sí	Sí	No	spam
541	Sí	Sí	No	spam
693	No	Sí	Sí	ham
782	No	No	No	ham
976	No	No	No	ham

Ejemplo de detección de Spam

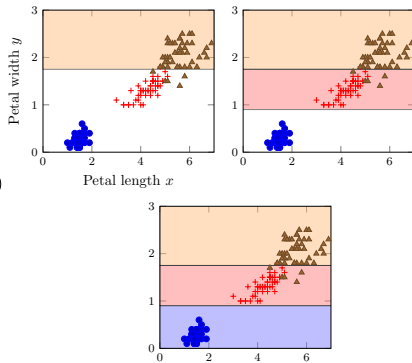
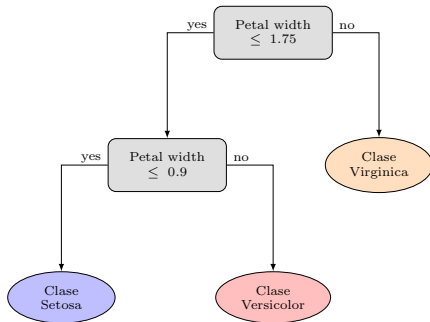


- ☐ Dos alternativas que conllevan a las mismas predicciones.
- ☐ ¿Cuál elegir?
- ☐ *La explicación más simple suele ser la más probable: Guillermo de Ockham*

Otro Ejemplo



Otras alternativas

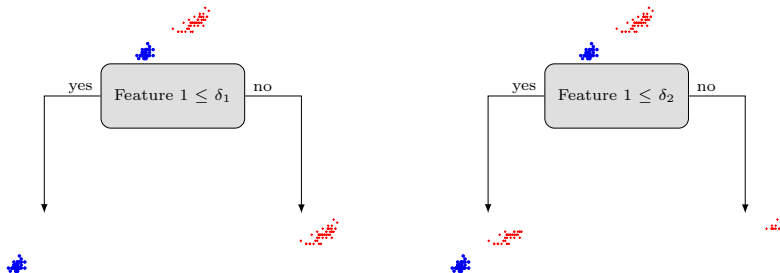


Proceso de entrenamiento I

Dado un conjunto de entrenamiento el proceso de entrenamiento se centra en dos decisiones:

Decisión 1 ¿Cuál característica debemos considerar en cada uno de los nodos?

Maximizar la pureza de cada nodo



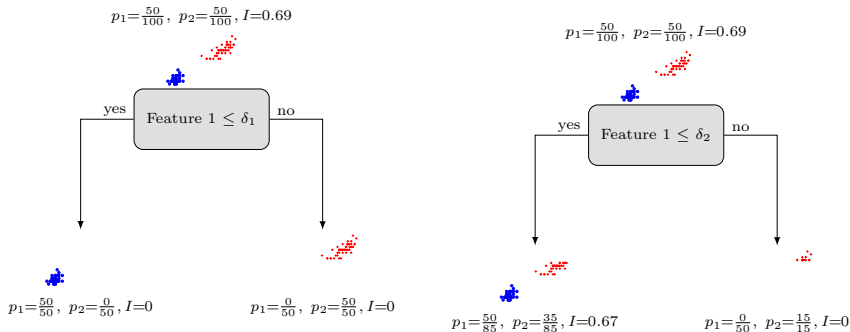
Proceso de entrenamiento II

Decisión 2 ¿Cuándo parar de realizar divisiones?

- ☐ Cuando el 100 % de las muestras en un nodo pertenecen a una única clase.
- ☐ Cuando al dividir un nodo se excede la profundidad máxima (hiperparámetro).
- ☐ Cuando la cantidad de muestras en un nodo es menor que un umbral (hiperparámetro).
- ☐ Cuando las mejoras en la pureza de los nodos no es mayor a un umbral (hiperparámetro).

Proceso de entrenamiento III: Valor Gini

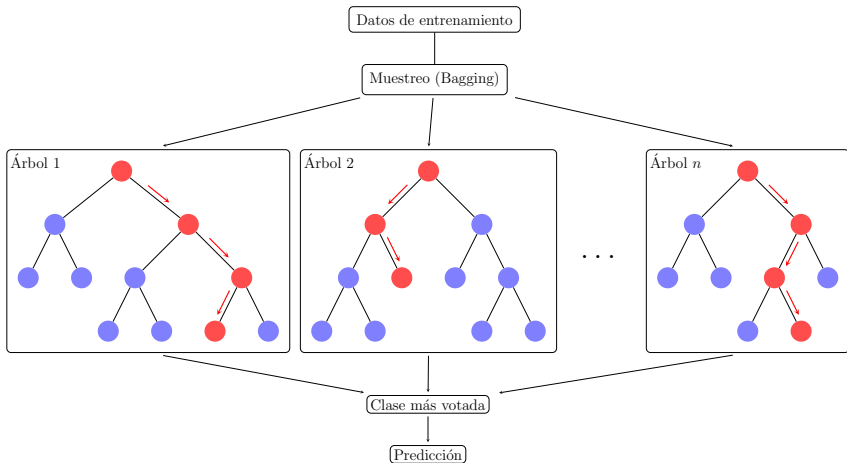
$$I = 1 - \sum_{k=1}^K p_k^2$$



Random Forest I

- El random forest consiste en utilizar Bagging sobre diversos árboles de decisión con los mismos hiper-parámetros.
- Por otro lado, también se hace un muestreo aleatorio sobre los atributos. De esta forma, no todos los árboles tienen acceso a las mismas variables.
- La predicción de una instancia nueva se hace a partir de la etiqueta más votada en los múltiples árboles entrenados.

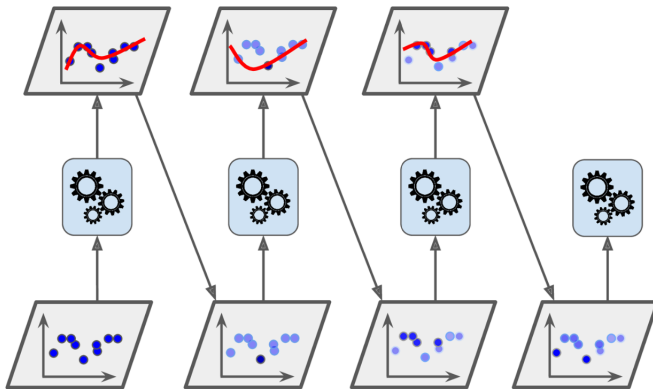
Random Forest II



Boosting - AdaBoost

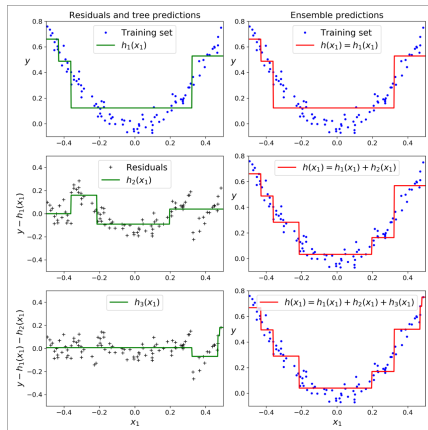
- ☐ Existen otros tipos de combinación de clasificadores, denominados Boosting.
- ☐ La idea principal es entrenar los modelos de forma secuencial de tal forma que el actual modelo corrige los errores de su predecesor.
- ☐ Los modelos más usados en este contexto son el AdaBoost y el Gradient Boost.
- ☐ En el AdaBoost el nuevo modelo tiene la capacidad de enfocar su atención en las muestras que su predecesor ajustó de forma incorrecta.
- ☐ Para clasificar una nueva muestra, se efectúa una votación por mayoría ponderada, donde las ponderaciones son proporcionales al rendimiento de cada clasificador en la etapa de entrenamiento.

AdaBoost¹



¹Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems.

Gradient Boost²



²Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems.

Agenda

1. Modelos Lineales

2. Máquinas de Vectores de Soporte

3. Modelos Basado en Árboles

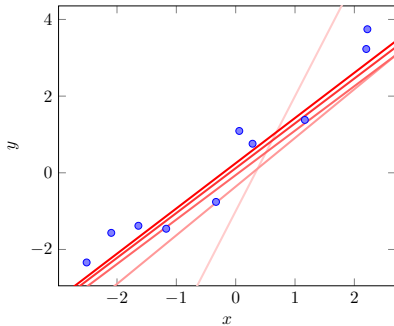
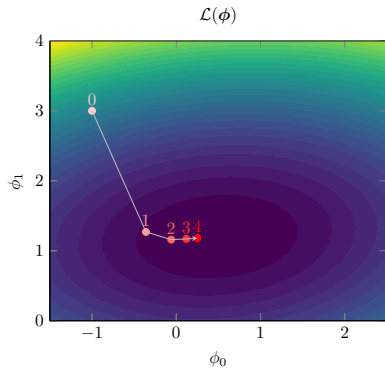
4. Redes Neuronales Profundas

5. Revisitando la Regresión Lineal

Agenda

1. Modelos Lineales
2. Máquinas de Vectores de Soporte
3. Modelos Basado en Árboles
4. Redes Neuronales Profundas
- 5. Revisitando la Regresión Lineal**

Entrenamiento de una Regresión Lineal



Funciones base I

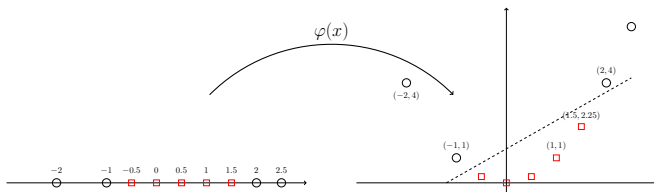
- El modelo lineal solo permite relaciones lineales entre x e y .
- Para capturar relaciones no lineales, una alternativa es utilizar funciones base $\varphi(x)$, que transforman los datos antes de aplicar la regresión lineal.
- Sin embargo, la relación con los parámetros w sigue siendo lineal:

$$f(x, w) = w_0 + \sum_{i=1}^M w_i \varphi_i(x) = w^\top \varphi(x)$$

- Donde $w \in \mathbb{R}^{M+1}$ y

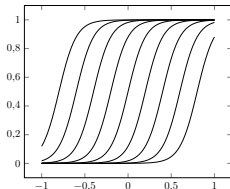
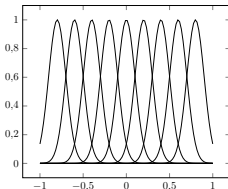
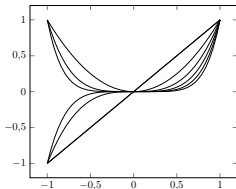
$$\varphi(x) = [1, \varphi_1(x), \dots, \varphi_M(x)]^\top$$

Funciones base II



- Considerar la transformación no lineal $\varphi(x) = [x, x^2]$.
- El teorema de **Cover** establece que si a un conjunto de datos se le aplica una transformación no-lineal, donde la dimensión de la transformación es lo suficientemente grande, los datos transformados tienen alta probabilidad de ser linealmente separable.

Ejemplos de funciones base



- Polinómicas:

$$\varphi_i(x) = x^i$$

- Exponenciales (Funciones de base Gaussiana):

$$\varphi_i(x) = \exp\left(-\frac{(x - \mu_i)^2}{2s^2}\right)$$

- Sigmoidales:

$$\varphi_i(x) = \sigma\left(\frac{x - \mu_i}{s}\right), \quad \sigma(a) = \frac{1}{1 + \exp(-a)}$$

Transformando los datos de entrada

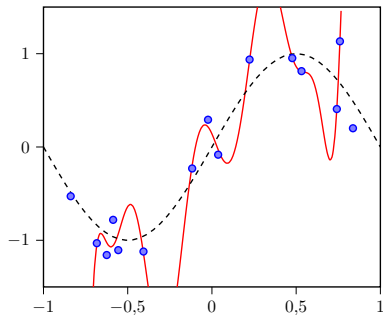
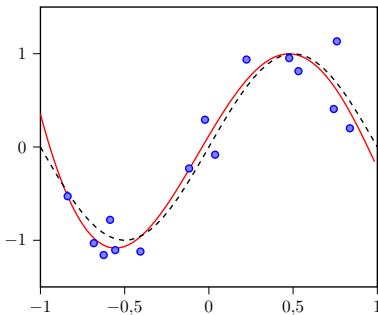
- Como ejemplo, suponer que se quiere predecir la función seno en el intervalo $(-1,1)$.
- Suponer que se quieren usar una transformación polinómica. Así, por cada dato se tiene

$$\varphi(x) = [1 \quad x \quad x^2 \quad x^3 \quad \dots \quad x^M]^\top$$

- Cada dato x se convierte a un espacio de más dimensiones $\varphi(x) \in \mathbb{R}^{M+1}$

Ejemplos con funciones base

- Ejemplo con $M = 5$ y $M = 13$



Funciones base: limitaciones

- Se debe tener cuidado puesto que la cantidad de funciones base puede limitar el rendimiento del modelo (sobre-ajuste).
- Por otro lado, el rendimiento también dependerá del tipo de funciones base que se elija:
 - Funciones Sigmoidales.
 - Funciones Gaussianas.
 - Funciones Polinomiales.

Redes Poco Profundas I

- Una alternativa es construir las funciones base de forma paramétrica.
- Los parámetros de estas funciones base se estiman junto a los parámetros del modelo a partir de gradiente descendente.
- De esta forma cada nueva característica se da por:

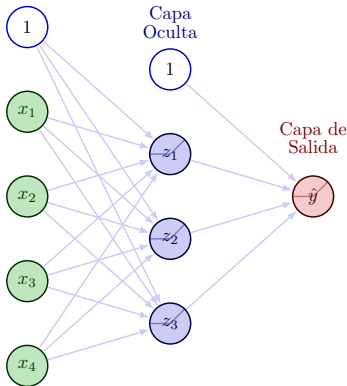
$$z_j = h \left(\sum_{p=1}^P w_{jp} x_p + w_{j0} \right),$$

donde $j \in \{1, \dots, M\}$. Además, $h(\cdot)$ se conoce como la función de activación.

- En la jerga del Deep Learning, cada $z_j(x)$ se conoce como una neurona o unidad.

Redes Poco Profundas II

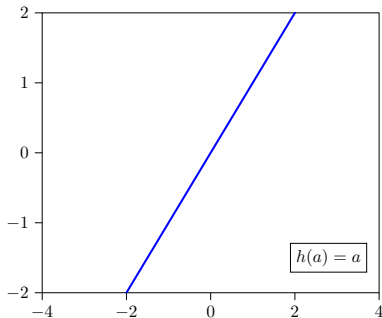
Capa de
Entrada



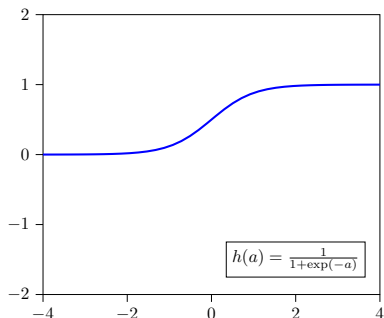
- Una red neuronal tiene tres tipos de capas: **la capa de entrada**, **la capa oculta** y **la capa de salida**.
- La capa de entrada está relacionada con los atributos de entrada.
- La capa oculta corresponde a una representación de los datos de entrada. Son las salidas de las M funciones base.
- Finalmente, la capa de salida se encarga de realizar predicciones de la red.
- En la gráfica un ejemplo con $m = 4$ y $M = 3$.

Funciones de Activación I

Función Lineal

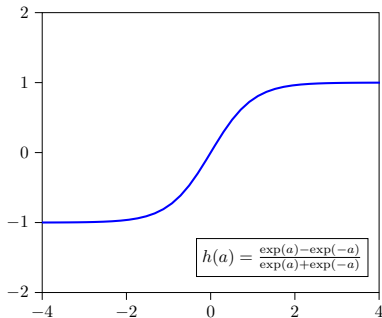


Función Sigmoidal

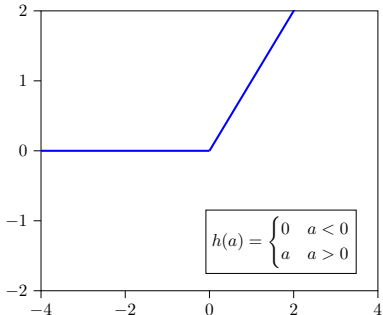


Funciones de Activación II

Función Tangente Hiperbólica

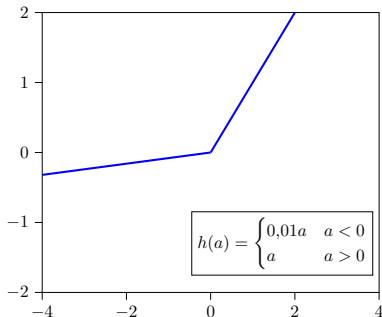


Función ReLU

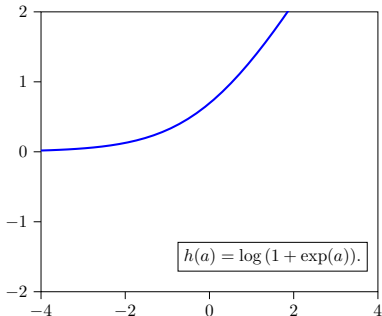


Funciones de Activación III

Función Leaky ReLU

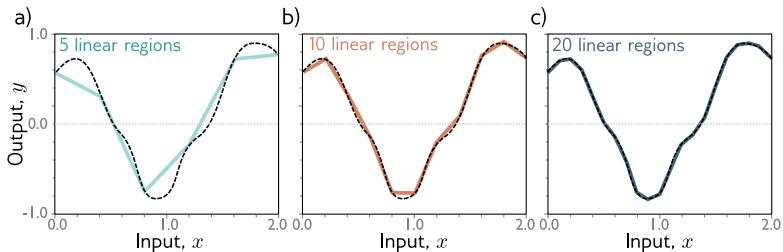


Función SoftPlus



Teorema de aproximación universal [1]

Con la cantidad suficiente de neuronas se puede describir cualquier función con una precisión arbitraria.

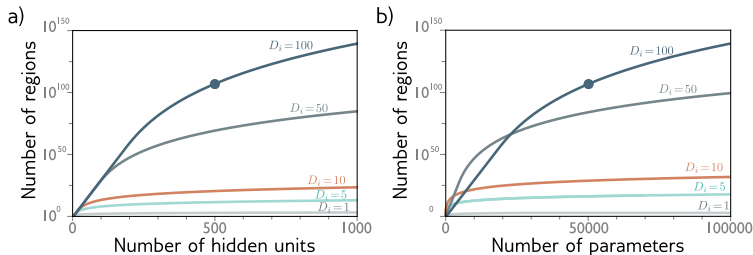


Teorema de aproximación universal [1]

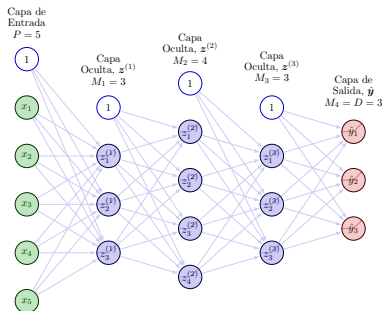
“A formal proof that, with enough hidden units, a shallow neural network can describe any continuous function on a compact subset of $\mathbb{R}^m$ to arbitrary precision”

Teorema de aproximación universal [1]

En general, las salidas de una red neuronal están compuestas a partir de múltiples regiones lineales. En la gráfica $D_i \rightarrow m$



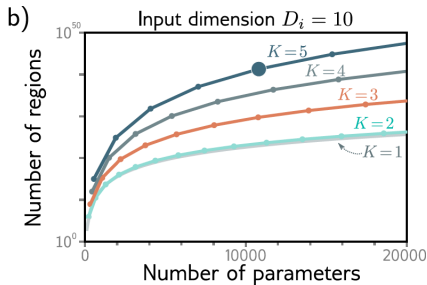
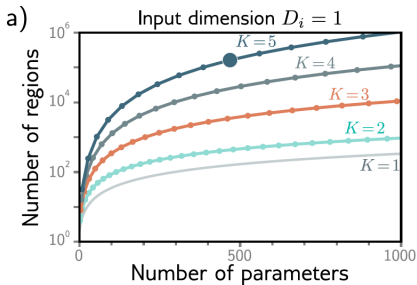
Red Neuronal de L capas



- Los conceptos vistos pueden ser aplicados a redes neuronales más profundas.
- Se muestra una red neuronal con $L = 3$ capas. En el conteo de las capas no se tiene en cuenta la capa de entrada.

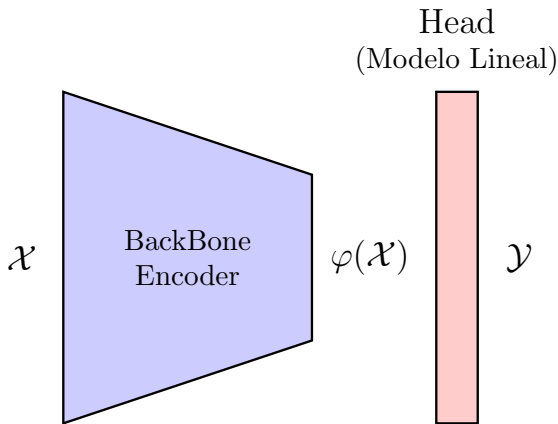
Shallow vs Deep I

- Número de regiones lineales por parámetro. En la gráfica $D_i \rightarrow m$ y $K \rightarrow L$

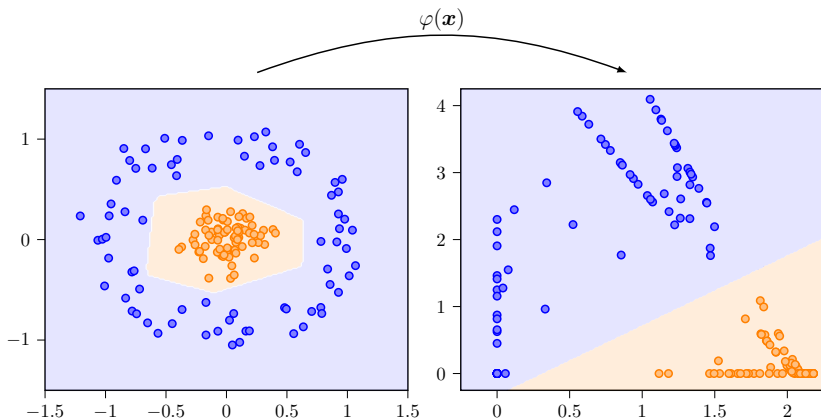


Representación de los datos I

- Representación: Las capas ocultas mejoran la representación de los datos. La capa de salida es un modelo lineal



Representación de los datos II



Referencias



Simon JD Prince.

Understanding Deep Learning.

MIT press, 2023.